Hamburg University of Technology
Institute of Mathematics
Chair Numerical Mathematics

Jens-Peter M. Zemke
Finn Sommer
Lars Stietz

Advanced Machine Learning
Winter Semester 2025/2026
Exercise Sheet 7
Week 8, 01.12.2025 – 05.12.2025

Objectives of this exercise sheet:

- Exercise 1 mimicks the implementation of convolutional layers in TensorFlow or PyTorch. Evaluating a convolutional layer is presented in a mathematical formula in Exercise 1 and in the lecture, ensure to look into it and try to understand it.
- Exercise 1 is the first step so understand Exercise 2. The good thing is that backprop is already implemented, so you can cheat a little bit by looking at it to better understand Exercise 2. Exercise 2 is a tough one if you look for a precise mathematical solution (which we will give you), but at the same time you know about feedforward, backprop, and update of convolutions in 1D and 2D (without the clutter of channels and multiple filters/inputs). The change lies in the additional summation over the channels and using the same filters at various places, so you have to add contributions at meaningful positions. Try to understand what happens if you add channels to the picture, then proceed to multiple filters, then to multiple images. Yes, in the end if written out in basic operations, you end up with a large amount of nested loops. But this is how it works in practice, no shortcut available here. The good point: if you understood and completed at least 30% of the task, you already learned a lot.
- Exercise 3 is concerned with pooling. Pooling is important in practice, but conceptually simple. It should not pose too much of a problem to understand it, but if you fail in coming up with a working implementation, no shame on you. The flatten layer maps a 4-tensor to a 2-tensor, what do you need to store to go back in the backprop? There you are, you started thinking about it, that is great.
- Exercise 4 is for the fun of it, I mean we are here to learn about ML and Deep Learning. This is an application of the stuff presented in the lecture on how to initialize weights (zero is bad; known is good $\rightsquigarrow$ this is the content of this exercise; random is best if you do not know anything, but be careful with the scaling). If I where you, I would be eager to test it out.

Remark: In order to use our library it has to be registered with `pip` for each exercise sheet. You can use the script `uninstall_amllib.py` to make sure the library version from the last exercise sheet is deleted. Then use `install_amllib.py` to register the library version for this exercise sheet. The script `install_amllib` registers the library as editable library, hence all changes are directly accessible. The script `uninstall_amllib.py` reverts the installation.

The star symbol ($\star$) indicates an optional task. These tasks are rather theoretical and some are intended to enhance your understanding of underlying mathematical concepts. Nonetheless, we recommend attempting them.

Homework:

Exercise 1: (5 points) level •○○ relevance •••
Complete the skeleton for the class `Conv2D` in `layers/conv2d.py`. It inherits from the base class `Layer` which requires the completion of the following methods:

- `feedforward(self, inputs)`: evaluate the convolutional layer

$$\mathbf{Z}(i,j,:,:) = b(j)\mathbf{E} + \sum_{k=1}^{c} \mathbf{A}(i,k,:,:) * \mathbf{F}(j,k,:,:), \quad \mathbf{A}_{\text{new}} = a(\mathbf{Z})$$

  and cache data for backpropagation. The matrix $\mathbf{E} \in \mathbb{R}^{z_h \times z_w}$ comprises ones.
- `update(self)`: update all parameters with `self.df` and `self.db` computed by the `backprop` method.

The method `backprop` is already implemented. Test your implementation with the unit tests provided in `test_conv2d.py`. You can run all tests by executing the script with python.

Exercise 2 ($\star$): (3 points) level ••○ relevance •○○
In standard implementations, e.g., in TensorFlow/Keras, due to Python's way of storing multiarrays (the last index moves fastest), (activated) minibatches $\mathbf{A}$ of $n \in \mathbb{N}$ images with $c \in \mathbb{N}$ channels of height $h \in \mathbb{N}$ and width $w \in \mathbb{N}$ are stored in the 4D tensor format

$$\mathbf{A} \in \mathbb{R}^{n \times c \times h \times w},$$

the so-called NCHW format. The filter bank $\mathbf{F}$ of $m \in \mathbb{N}$ filters (m feature maps) of width $f_w \in \mathbb{N}$ and height $f_h \in \mathbb{N}$ that sums over the aforementioned channels c is stored in the 4D tensor format

$$\mathbf{F} \in \mathbb{R}^{m \times c \times f_h \times f_w}.$$

For every feature map there exists a bias, thus the bias is a vector $\mathbf{b} \in \mathbb{R}^m$.

Derive the backpropagation and kernel update of the feedforward step

$$\mathbf{Z}(i,j,:,:) = b(j)\mathbf{E} + \sum_{k=1}^{c} \mathbf{A}(i,k,:,:) * \mathbf{F}(j,k,:,:), \quad \mathbf{A}_{\text{new}} = a(\mathbf{Z})$$

for the valid convolution and given activation function a by hand, where

$$\boldsymbol{\Delta} := \frac{\partial C}{\partial \mathbf{Z}} \in \mathbb{R}^{n \times m \times z_h \times z_w}, \quad z_h = h - f_h + 1, \quad z_w = w - f_w + 1$$

is given. The matrix $\mathbf{E} \in \mathbb{R}^{z_h \times z_w}$ comprises ones.

Exercises (in class):

Exercise 3: (3 points) level ••• relevance ••○
Complete the classes `MaxPool2D` and `Flatten` in `layers/pool2d.py` and `layers/flatten.py`, respectively. Both classes inherit from the base class `Layer` which requires the completion of the following methods:

a) `MaxPool2D.feedforward(self, a)`: apply max pooling to an input, cache all data necessary for the backpropagation.
b) `MaxPool2D.backprop(self, delta)`: use the information cached by the previous call `MaxPool2D.feedforward` to backpropagate.
c) `Flatten.feedforward(self, a)`: reshape the input, cache all data necessary for backpropagation.
d) `Flatten.backprop(self, delta)`: use the data cached by the previous call to the method `Flatten.feedforward` to backpropagate.

Test your implementation with the unit tests provided in `test_maxpool` and `test_flatten.py`. You can run all tests by executing the scripts with python.

Exercise 4: (1+1+1+1+0 points) level ●○○ relevance ●●○

Run the TensorFlow script `skeleton/exercises/transferlearning.py` that implements a CNN based on transfer learning for classifying images of cats and dogs. You might want to decrease the number of epochs if your hardware is too restricted. Afterwards answer the following questions:

a) What does the function `get_cats_and_dogs` do? Explore the new directory `datasets`.
b) In the function `get_base_model`: what does `include_top=False` do? What is the effect of `weights='imagenet'`? What is the result of `base_model.trainable = False`?
c) Look up `tf.keras.losses.BinaryCrossentropy` in the TensorFlow/Keras documentation. Why do we use `BinaryCrossentropy(from_logits=True)`?
d) Look into the new file `transfer_model.pdf` and at the output of `model.summary()`. What is a global average pooling layer? How many parameters are we training?
e) What are pro and con of using this transfer learning approach?